

Лабораторная работа № 8 : Элементы криптографии. Шифрование различных исходных текстов одним ключом

Эйвази Мани

7.октябрь.2026

Содержание

1	Отчёт по лабораторной работе № 8	2
2	Цели и задачи работы	2
2.1	Цель лабораторной работы	2
2.2	Задачи	2
3	Процесс выполнения лабораторной работы	3
3.1	Создание программы main.py	3
3.2	Запуск программы	4
3.3	Аналитическое обоснование атаки без ключа	5
4	Содержание отчёта	8
5	Выводы по проделанной работе	8
5.1	Выводы	8
6	Цель работы достигнута:	9

1 Отчёт по лабораторной работе № 8

Элементы криптографии. Шифрование различных исходных текстов одним ключом

2 Цели и задачи работы

2.1 Цель лабораторной работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

2.2 Задачи

1. Два текста кодируются одним ключом (однократное гаммирование). Требуется, не зная ключа и не стремясь его определить, прочитать оба текста. Необходимо разработать приложение, позволяющее шифровать и дешифровать тексты P1 и P2 в режиме однократного гаммирования. Приложение должно определить вид шифротекстов C1 и C2 обоих текстов P1 и P2 при известном ключе. Необходимо определить и выразить аналитически способ, при котором злоумышленник может прочитать оба текста, не зная ключа и не стремясь его определить.

Исходные данные: $P1 = \text{На Ваши исходящий 1204}$ $P2 = \text{В Северный филиал Банка}$
Ключ K (20 байт): 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54

3 Процесс выполнения лабораторной работы

3.1 Создание программы main.py

Действие: В директории /lab_08 создать файл main.py с реализацией шифрования двух текстов одним ключом.

cd ~/lab_08 nano main.py

Листинг программы main.py:

```
root@localhost:~/lab_08# cat program
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

def xor_bytes(a: bytes, b: bytes) -> bytes:
    """Побитовый XOR двух последовательностей байт одинаковой длины."""
    return bytes(x ^ y for x, y in zip(a, b))

def to_hex(data: bytes) -> str:
    return ' '.join(f'{b:02X}' for b in data)

def text_to_bytes(text: str) -> bytes:
    return text.encode('cp1251')

def bytes_to_text(data: bytes) -> str:
    return data.decode('cp1251', errors='replace')

def main():

    P1_text = "На Ваши сходящий 1204"
    P2_text = "В Северный филиал Банка"

    key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54"
    K = bytes(int(x, 16) for x in key_hex.split())

    P1 = text_to_bytes(P1_text)
    P2 = text_to_bytes(P2_text)

    print("=== Исходные данные ===")
    print(f"P1 = {P1_text}")
    print(f"P2 = {P2_text}")
    print(f"K = {to_hex(K)}")
    print(f"Длина K = {len(K)} байт")
    print(f"Длина P1 = {len(P1)} байт")
    print(f"Длина P2 = {len(P2)} байт")
    print()

    # --- Шифрование ---
    C1 = xor_bytes(P1, K)
    C2 = xor_bytes(P2, K)

    print("=== Шифротексты ===")
    print(f"C1 = {to_hex(C1)}")
    print(f"C2 = {to_hex(C2)}")
```

```

# --- Шифрование ---
C1 = xor_bytes(P1, K)
C2 = xor_bytes(P2, K)

print("=== Шифротексты ===")
print(f"C1 = {to_hex(C1)}")
print(f"C2 = {to_hex(C2)}")
print()

# --- Проверка: дешифрование ключом ---
P1_dec = xor_bytes(C1, K)
P2_dec = xor_bytes(C2, K)

print("=== Дешифрование ключом (проверка) ===")
print(f"P1' = {bytes_to_text(P1_dec)}")
print(f"P2' = {bytes_to_text(P2_dec)}")
print()

# --- Атака без ключа ---
# Если известен P1, то P2 = (C1 xor C2) xor P1
X = xor_bytes(C1, C2) # = P1 xor P2
print("=== C1 xor C2 = P1 xor P2 ===")
print(f"X = {to_hex(X)}")
print()

# Восстановим P2, зная P1
P2_recovered = xor_bytes(X, P1)
print("=== Восстановление P2 при известном P1 (без ключа) ===")
print(f"P2 (восстановленный) = {bytes_to_text(P2_recovered)}")
print()

# Восстановим P1, зная P2
P1_recovered = xor_bytes(X, P2)
print("=== Восстановление P1 при известном P2 (без ключа) ===")
print(f"P1 (восстановленный) = {bytes_to_text(P1_recovered)}")
print()

# --- Демонстрация пошагового восстановления ---
print("=== Пошаговое восстановление P2 по шаблону P1 ===")
known_P1 = "НвВашисходящий1204"
known_bytes = text_to_bytes(known_P1)
# Восстанавливаем P2 там, где известен P1
partial = bytearray(len(P2))
for i in range(min(len(known_bytes), len(X))):
    partial[i] = X[i] ^ known_bytes[i]
print(f"Частично восстановленный P2: {bytes_to_text(bytes(partial))}")

if __name__ == "__main__":
    main()
root@localhost:~/lab_08#

```

Рис. 3,4,5 — python

3.2 Запуск программы

Сделать файл исполняемым и запустить.

```

root@localhost:~/lab_08# ls
program  report
root@localhost:~/lab_08# nano main.py
root@localhost:~/lab_08# ls
main.py  program  report
root@localhost:~/lab_08# chmod +x main.py
root@localhost:~/lab_08# ls
main.py  program  report
root@localhost:~/lab_08# ./main.py
=== Исходные данные ===
P1 = НаВашисходящий1204
P2 = ВСеверныйфилиалБанка
K = 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 08 B2 70 54
Длина K = 20 байт
Длина P1 = 20 байт
Длина P2 = 20 байт

=== Шифротексты ===
C1 = C8 EC D5 9F F6 A6 C6 27 7A F4 F6 D7 CA BE 11 3A 3A 80 40 60
C2 = C7 DD F2 9D EB BE DA 29 7D E4 E1 C5 CA B7 14 09 EB 5F 9A B4

=== Дешифрование ключом (проверка) ===
P1' = НаВашисходящий1204
P2' = ВСеверныйфилиалБанка

=== C1 xor C2 = P1 xor P2 ===
X = 0F 31 27 02 1D 18 1C 0E 07 10 17 12 00 09 05 33 D1 DF DA D4

=== Восстановление P2 при известном P1 (без ключа) ===
P2 (восстановленный) = ВСеверныйфилиалБанка

=== Восстановление P1 при известном P2 (без ключа) ===
P1 (восстановленный) = НаВашисходящий1204

=== Пошаговое восстановление P2 по шаблону P1 ===
Частично восстановленный P2: ВСеверныйфилиалБанка
root@localhost:~/lab_08#

```

Рис. 7

3.3 Аналитическое обоснование атаки без ключа

Действие: Показать, что злоумышленник может прочитать оба текста, не зная ключа. Аналитический вывод:

Из формул шифрования:

$$C_1 = P_1 \oplus K, \quad C_2 = P_2 \oplus K$$

Выполним XOR обоих шифротекстов:

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$$

Ключ K сокращается, так как $K \oplus K = 0$.

Таким образом, злоумышленник, зная $C_1 \oplus C_2$, может восстановить:

- $P_2 = (C_1 \oplus C_2) \oplus P_1$, если известен P_1 ,
- $P_1 = (C_1 \oplus C_2) \oplus P_2$, если известен P_2 .

Пошаговое восстановление:

Если один из текстов является шаблоном (имеет фиксированный формат), злоумышленник может:

1. Использовать известные символы шаблона для восстановления соответствующих символов второго текста.
2. На основе логики сообщения P_2 предположить новые символы.
3. Повторять шаги 1–2, постепенно восстанавливая всё сообщение.

Пример: Если известно, что P_1 — шаблон `наВашисходящий1204`, то:

- $P_2 = X \oplus P_1$, где $X = C_1 \oplus C_2$.
- Результат: `ВСеверныйфилиалБанка`.

4. Ответы на контрольные вопросы

1. Как, зная один из текстов (P1 или P2), определить другой, не зная при этом ключа?

- Вычислить $X = C_1 \oplus C_2 = P_1 \oplus P_2$. Затем $P_2 = X \oplus P_1$ или $P_1 = X \oplus P_2$.

2. Что будет при повторном использовании ключа для шифрования текста?

Это нарушает условие однократного использования. Злоумышленник может вычислить $P_1 \oplus P_2$ и, используя шаблоны или логику сообщений, восстановить оба текста без знания ключа.

3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов?

- Каждый текст шифруется одним и тем же ключом: $C_1 = P_1 \oplus K$, $C_2 = P_2 \oplus K$.

4. Перечислите недостатки шифрования одним ключом двух открытых текстов. Нарушение условия однократного использования ключа. Возможность восстановления обоих текстов без знания ключа. Снижение криптостойкости. Ключ становится уязвимым для атаки.

5. Перечислите преимущества шифрования одним ключом двух открытых текстов. Экономия ключевого материала. Простота реализации. Возможность использования одного ключа для нескольких сообщений (при соблюдении дополнительных мер безопасности).

4 Содержание отчёта

- Титульный лист
- Цель работы
- Описание процесса выполнения с командами и снимками экрана
- Листинг программы *main.py*
- Результаты работы программы
- Аналитическое обоснование атаки без ключа
- Ответы на контрольные вопросы
- Выводы, согласованные с целью работы

5 Выводы по проделанной работе

5.1 Выводы

В ходе лабораторной работы я:

1. Разработал приложение *main.py* на языке Python, реализующее шифрование и дешифрование двух текстов одним ключом в режиме одноразового гаммирования.
2. Зашифровал два текста P_1 и P_2 одним ключом K :

$$\circ C_1 = P_1 \oplus K$$

$$\circ C_2 = P_2 \oplus K$$

3. Показал аналитически и практически, что злоумышленник может восстановить оба текста, не зная ключа:

$$\circ C_1 \oplus C_2 = P_1 \oplus P_2$$

$$\circ P_2 = (C_1 \oplus C_2) \oplus P_1$$

$$\circ P_1 = (C_1 \oplus C_2) \oplus P_2$$

4. Продемонстрировал пошаговое восстановление P_2 по шаблону P_1 .
5. Убедился, что повторное использование ключа для шифрования разных текстов нарушает условие одноразовости и снижает криптостойкость.
6. Ключевой вывод: Шифрование двух текстов одним ключом в режиме одноразового гаммирования позволяет злоумышленнику восстановить оба текста без знания ключа, используя свойство

$$C_1 \oplus C_2 = P_1 \oplus P_2.$$

7. Это демонстрирует важность одноразового использования ключа для обеспечения абсолютной стойкости шифра.

6 Цель работы достигнута:

освоено практическое применение режима одноразового гаммирования для кодирования различных исходных текстов одним ключом.

7. Краткая сводка команд

`bash`

Переход в рабочую директорию

`cd ~/lab_08`

Создание программы

`nano main.py`

(вставить листинг программы)

Сделать исполняемым и запустить

`chmod +x main.py`

`./main.py`

Просмотр файла cipher

`cat cipher`